AI 211

Computational Linear Algebra for AI

Problem Set

University of the Philippines Diliman

Master of Engineering in Artificial Intelligence

Submitted by:

Deomary Angelo B. Franco

Submitted to:

Jaymar Soriano, D.Sc.

June 2024

1. Eigenfaces and Face reconstruction
   a. Observe what features are represented by the first few eigenfaces.

      To visually understand what is present for the first few eigenfaces, the eigenfaces at the middle and the eigenfaces at the end were also plotted. Based on the plots, it can be observed that the first few eigenfaces are ghastly faces that form the main structure of the face and captures variation in lighting of the original faces while the eigenfaces on the middle and end seem to just look like "noise data" as these are the captured minor variations in the data set that are not crucial to represent overall facial representation.

Figure 1: First k=36 eigenfaces displaying the important aspects of the faces.

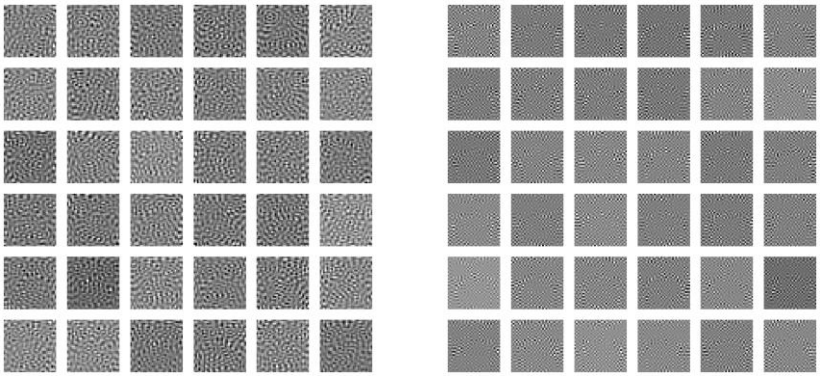


Figure 2: The left 36 images are the 400-435 eigenfaces while the right 36 images are the 900-935 eigenfaces. It can be noticed that both sets contain very little information about the original dataset.

b. Discuss how dimensionality reduction is performed and what is the rationale of doing dimensionality reduction.

Based on viewing the eigenfaces, only the first few resemble faces while as you move forward, the images become random data. These random data are believed to only be errors that can be neglected. With this, we reduce the effects of noise and less data to store since for face recognition systems, the system does not need to remember all the faces. It just needs to store the key eigenfaces since these eigenfaces serve as the basis for all other faces. By removing some negligible basis (noise data), we can further reduce the needed data to reconstruct all the faces but with minimal loss of information.

c. Discuss how compression is achieved using SVD. Reconstruct the faces using the first k eigenfaces and compute the error of reconstruction averaged over all the faces. Plot the average reconstruction error versus k from 1 to 100.

SVD performs the compression such that any system utilizing the face images will not need to memorize the whole data set (in this case the 5000 x 1024 data) but instead would only need a fewer data.

On reconstruction, if only data related to the first k-eigenfaces are considered, shape of data is shown as:

$$U\ shape\ =\ 5000 \times\ k$$

$$S\ shape\ =\ k \times\ k$$

$$V^T\ shape = k \times 1024$$

The only required data to be stored in memory is:

$$Total\ data = 5000k + k^2 + 1024k$$

And for $k = 100$,

$$Total\ data = 5000 \times 100 + 100^2 + 1024 \times 100$$

$$Total\ data = 612{,}400 < 5{,}120{,}000 = 5000 \times 1024$$

This is significantly lighter and when original data set is required, reconstruction is readily available by matrix multiplication on-the-fly instead of storing the whole data set on memory.

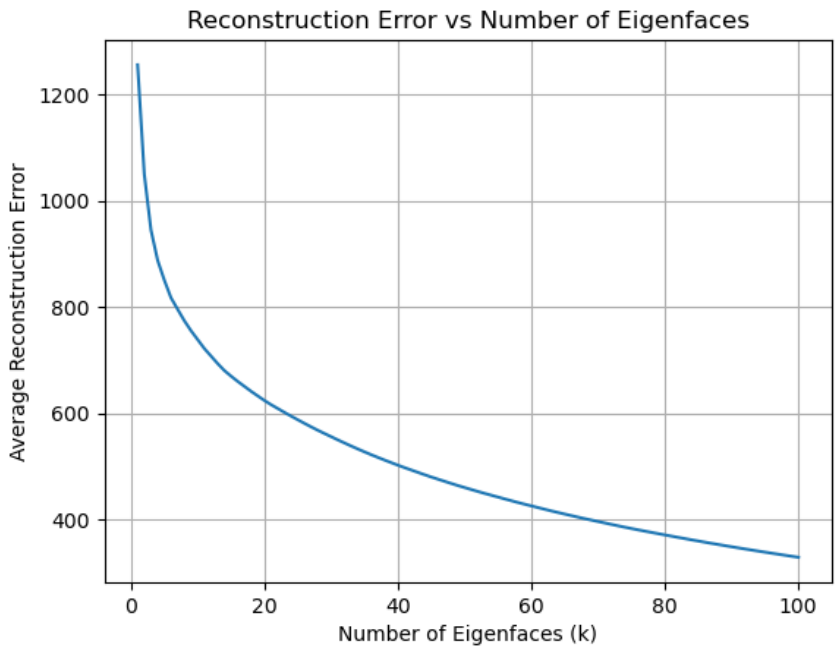


Figure 3: Plot of average reconstruction error vs first k eigenfaces. As k increases, average reconstruction error decreases, however the rate of decrease also significantly decreases. This implies that selecting more eigenfaces has diminishing returns and will just require higher memory to store more data.

Figure 4: Shown here is the reconstruction of the original dataset using only U, S, and Vt but containing only the first k=100 eigenfaces. It is observed that minimal information is lost but with significant savings on memory storage. This tradeoff is preferable for most use cases.

2. Nonlinear vs Linearized Least Squares Regression
   a. Given the provided data, determine v_{max} and K_m using nonlinear least squares (Gauss-Newton).

Model:
Let $\hat{v}_0$ be a nonlinear model that predicts the initial velocity

$$\hat{v}_0 = \frac{v_{max}}{1 + \frac{K_m}{C}}$$

Let θ be a vector that holds the parameters of $\hat{v}_0$. The goal is to calculate the optimized values of θ such that the sum of the square of errors is minimized.

$$\theta = \begin{pmatrix} v_{max} \\ K_m \end{pmatrix}$$

Residual:
Let r be the difference between predicted initial velocity and measured initial velocity.

$$r(C;\ \theta) = \hat{v}_0 - v_0$$
$$r(C;\ \theta) = \frac{v_{max}}{1 + \frac{K_m}{C}} - v_0$$
$$r(C;\ \theta) = \frac{C v_{max}}{C + K_m} - v_0$$

Jacobian of Residuals:
Calculate the Jacobian of the residuals where r is parameterized by all elements of C. Here J is a 5 by 2 matrix since there are 5 elements of C and 2 parameters.

$$J\big(r(\theta)\big) = \left(\left(\frac{\partial r(C)}{\partial v_{max}}\right)\left(\frac{\partial r(C)}{\partial K_m}\right)\right)$$
$$J\big(r(\theta)\big) = \left(\left(\frac{C}{C + K_m}\right)\left(-\frac{C v_{max}}{(C + K_m)^2}\right)\right)$$

Using Gauss-Newton method for nonlinear models, the main algorithm is to iteratively solve for θ until the difference of $\theta^{(k+1)}$ and $\theta^{(k)}$ is within tolerance. Note here that $J\big(r(\theta)\big)$ is rewritten as $J_r(\theta)$.

$$\theta^{(k+1)} = \theta^{(k)} - \left[H\big(\theta^{(k)}\big)\right]^{-1} J_r^T\big(\theta^{(k)}\big) r\big(\theta^{(k)}\big)$$

Here, the Hessian of theta $H(\theta)$ is approximated as $J_r^T(\theta) J_r(\theta)$.

$$\theta^{(k+1)} = \theta^{(k)} - \left[J_r^T\big(\theta^{(k)}\big) J_r\big(\theta^{(k)}\big)\right]^{-1} J_r^T\big(\theta^{(k)}\big) r\big(\theta^{(k)}\big)$$

By initializing $\theta^{(0)}$ with a random vector and after some iterations, θ converges to

$$\theta = \begin{pmatrix} 0.085857 \\ 6.561894 \end{pmatrix}$$

Where it is assumed that the initialized vector does not cause any matrix singularities.

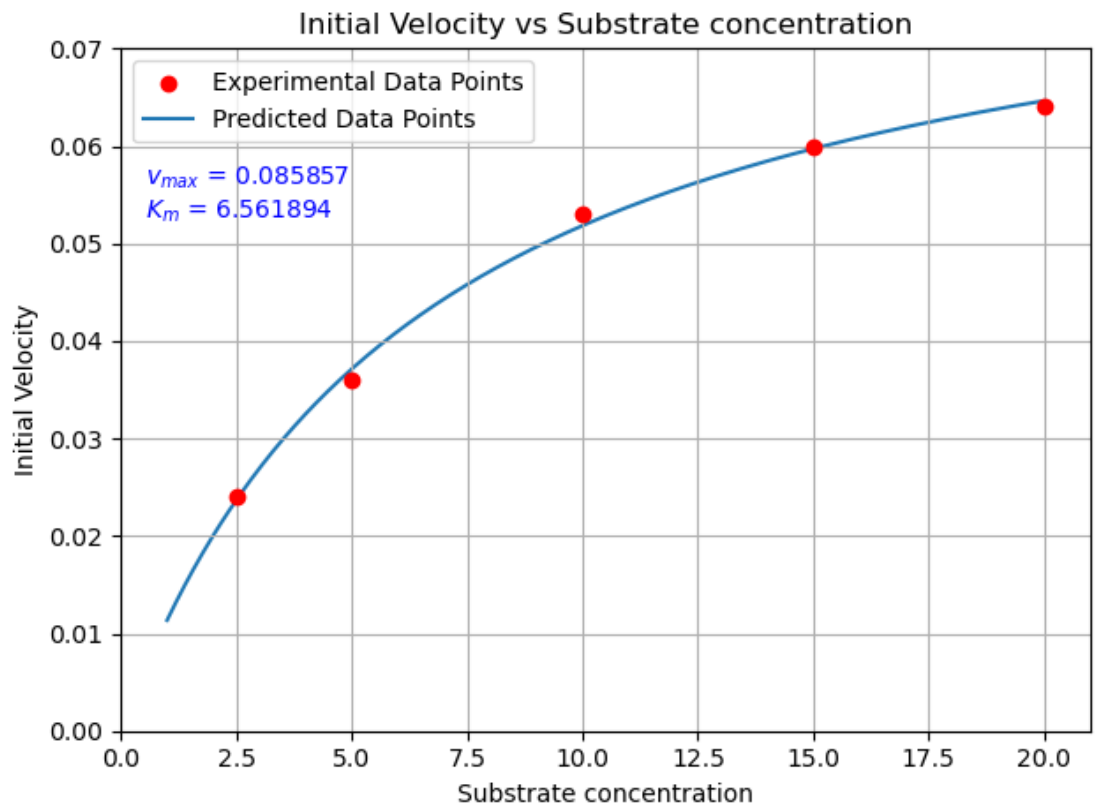


Figure 5: It can be observed that the predicted data points accurately fit the experimental data that it is given using Gauss-Newton's nonlinear method.

b. Alternatively, nonlinear curve-fitting is avoided by rewriting the equation in linear form:
Lineweaver

$$\frac{1}{v_0} = \frac{1}{v_{max}} + \frac{K_m}{v_{max}}\frac{1}{C}$$

Dixon

$$\frac{C}{v_0} = \frac{K_m}{v_{max}} + \frac{1}{v_{max}}C$$

Eadie

$$v_0 = v_{max} - \frac{K_m v_0}{C}$$

Using the same data, determine v_{max} and K_m using linear least squares from each of these forms.

Linear models will be assumed here wherein they follow the pattern

$$y = A\theta$$

Such that the solution is

$$\theta = A^+ y$$

Where A^+ is the pseudoinverse using Singular Value Decomposition.

$$A^+ = V\Sigma^+ U^T$$

$$\theta = V\Sigma^{+}U^{T}y$$

Note that within the equation $\theta = V\Sigma^{+}U^{T}y$, no iterations were performed. However, iterative steps are required when solving the SVD since the algorithm uses repetitive Householder QR decomposition.

Let us summarize the 3 equations such that they follow a linear model.

	Lineweaver	Dixon	Eadie
y	$\frac{1}{v_0}$	$\frac{C}{v_0}$	v_0
A	$\begin{pmatrix} 1 & \frac{1}{C} \end{pmatrix}$	$\begin{pmatrix} 1 & C \end{pmatrix}$	$\begin{pmatrix} 1 & \frac{-v_0}{C} \end{pmatrix}$
θ	$\begin{pmatrix} \frac{1}{v_{max}} \\ \frac{K_m}{v_{max}} \end{pmatrix}$	$\begin{pmatrix} \frac{K_m}{v_{max}} \\ \frac{1}{v_{max}} \end{pmatrix}$	$\begin{pmatrix} v_{max} \\ K_m \end{pmatrix}$

Presented below are the results of solving θ. v_{max} and K_m are then solved accordingly.

	Gauss-Newton (Nonlinear)	Lineweaver	Dixon	Eadie
θ	$\begin{pmatrix} 0.085857 \\ 6.561894 \end{pmatrix}$	$\begin{pmatrix} 11.8003 \\ 75.4314 \end{pmatrix}$	$\begin{pmatrix} 75.8081 \\ 11.7180 \end{pmatrix}$	$\begin{pmatrix} 0.08558 \\ 6.5155 \end{pmatrix}$
v_{max}	0.085857	0.084743	0.085339	0.085581
K_m	6.561894	6.392313	6.469373	6.515470

c. Compare the results in (b) with those obtained using nonlinear least squares in (a). Why do they differ? Which is/are closest to those determined in (a)? Why?

Shown below are the percentage error of the 3 different linear methods when compared to the nonlinear Gauss-Newton. The differences are due to the transformation of a nonlinear model to a linear model. The weights or parameters that are being optimized for the linear models are different than that of the Gauss-Newton. They also have different cost functions that they are attempting to minimize, hence the difference in results.

Gauss-Newton Comparison	Gauss-Newton (Nonlinear)	Lineweaver	Dixon	Eadie
$\left\lvert\frac{v_{max,NL} - v_{max}}{v_{max,NL}}\right\rvert \%$	0.00	1.30	0.60	0.32
$\left\lvert\frac{K_{m,NL} - K_m}{K_{m,NL}}\right\rvert \%$	0.00	2.58	1.41	0.71

The model nearest to the nonlinear model is the Eadie model since they try to optimize the same weights however, they minimize different cost functions. The Eadie model incorporates v_0 inside the model itself and since it is known that the model has some error when compared to the measured experimental results, hence that also adds to the overall error.

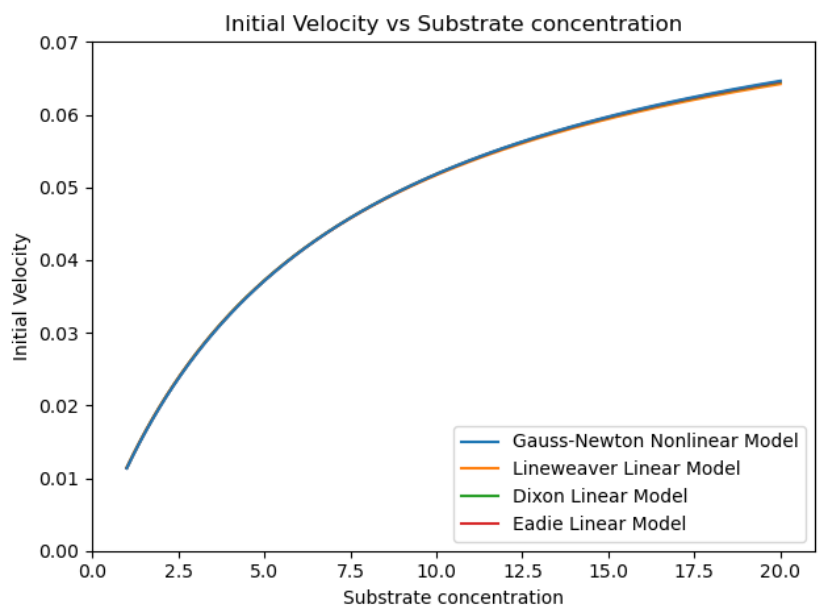


Figure 6: Based on visual observation, the 3 linear models have very little error when compared to the nonlinear model used. The plots are indistinguishable from each other.

3. MNIST Handwritten Digit Classification Using ANN

The code for the Artificial Neural Network can be found in the appendix. The goal is to create a model such that it can predict what number the images portray. The model is a relatively simple model such that it contains 3 layers namely the input layer, one hidden layer, and the output layer. The input layer has 400 + 1 nodes since it includes the bias, then 25 + 1 nodes for the hidden layer, and the output layer which obviously has 10 nodes since this is what is used to predict the results for this case.

The data is split into X and y where X contains the image data and y is the label for this classification problem. In this case, y is re-encoded such that the values now range from [0, 9] instead of [1, 10] which is the original. This is convenient since the "zero" images are found at the first 500 rows of the dataset which can be interpreted as the 0th index and the "nine" images are found at the last rows which is the 9th index. This is much easier to one-hot-encode for comparison later.

For the first instance of training the model, the weights are initialized randomly by using $np.random.rand(*\ size)$ but cost changes per epoch have negligible changes. Hence, the idea to initialize it using $np.random.rand(*\ size)$ and then subtracted by 0.5 so that the weights will be centered around zero and ranges from [-0.5, 0.5]. The activation function used here to nonlinearized the model is the sigmoid function.

Let us define the following:
a_1 → The original data set appended to the left with ones since this will serve as the multiplier to bias found on the matrix of weights. Size is 5000 x 401.
θ_1 → A matrix containing the weights that are multiplied to a_1. Size is 25 x 401.
z_2 → The matrix product $a_1\theta_1^T$. The element ij of z_2 is the dot product of ith row of a_1 and j^{th} row of θ_1. Size is 5000 x 25.
a_2 → This is the "squeezed" matrix equivalent of z_2 due to the activation function which is the sigmoid function. The activation function normalizes the weights to the range of (0, 1). Note that the activation function is applied elementwise. A column of ones is then appended to the left, same reason to that of a_1. Size is 5000 x 26.
θ_2 → A matrix containing the weights that are multiplied to a_2. Size is 10 x 26.
z_3 → The matrix product $a_2\theta_2^T$. The element ij of z_3 is the dot product of ith row of a_2 and j^{th} row of θ_2. Size is 5000 x 10.
a_3 → The "squeezed" equivalent of z_3. The result here is what determines what the classification of the digit image is. Size is 5000 x 10.

Shown below is a simple diagram portraying the design of the model.

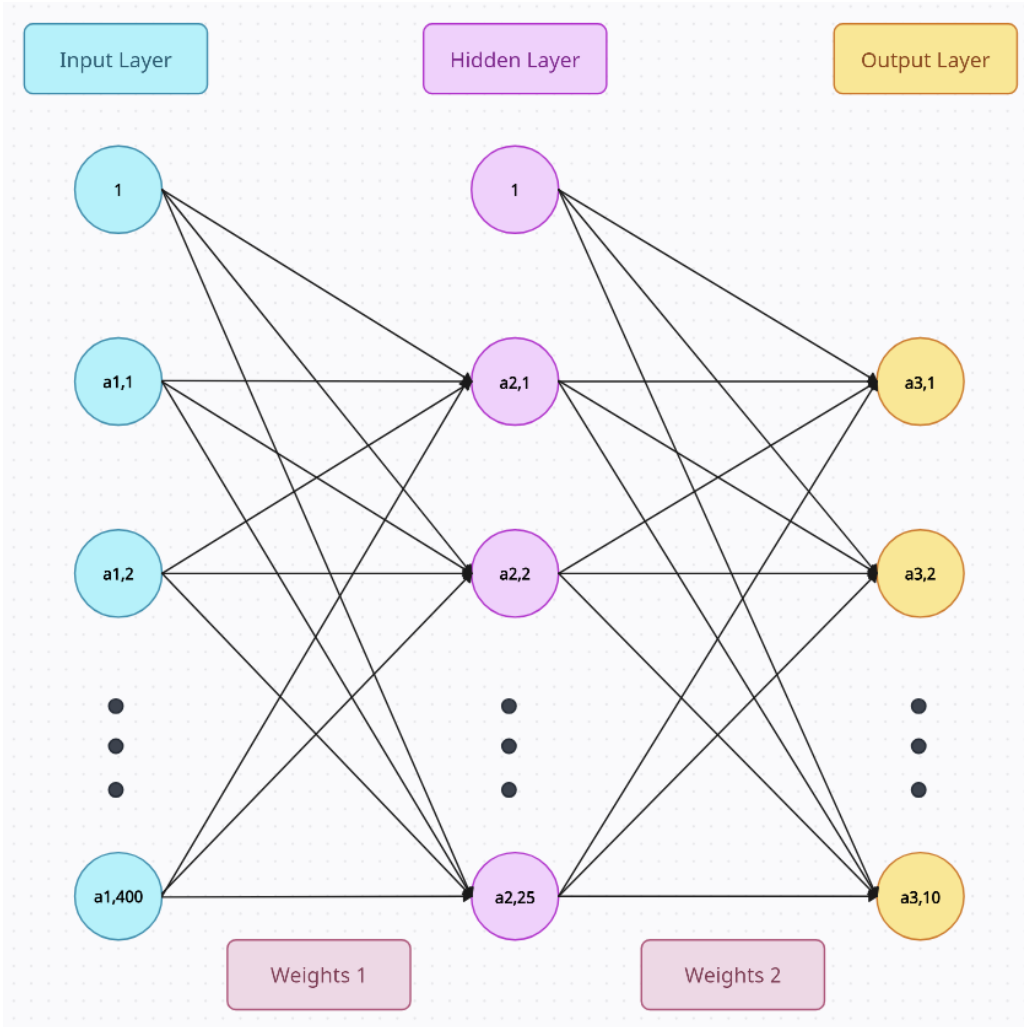


Figure 7: A diagram of the Artificial Neural Network with 1 input layer, 1 hidden layer, and 1 output layer. The model has 2 sets of weights.

Forward Propagation:

During forward propagation, the whole neural networks can be interpreted as a series of matrix multiplication and transformation by activation functions. The activation function σ is introduced for nonlinearity because if there were no activation function, the model is ultimately composite functions of linear equations which in turn is still a linear equation.

$$z_2 = a_1\theta_1^T + b_1$$
$$a_2 = \sigma(z_2)$$
$$z_3 = a_2\theta_2 + b_2$$
$$a_3 = \sigma(z_3)$$

By combining all the above,

$$a_3(\theta_1, \theta_2) = \sigma([\sigma(a_1\theta_1^T + b_1)]\theta_2^T + b_2)$$

Due to the sigmoid activation function, a_3 values must range between (0, 1).

Loss function:
Our label data or y data is the target. By one-hot-encoding y, it becomes a 5000 x 10 matrix where the value is one corresponding to the index of its value e.g. "six" images have a value of 1 on the 6th index and zero at all other indices.

The loss function here is then defined as the sum of the squares of the elementwise difference of a_3 and y.

$$J(\theta_1, \theta_2) = \frac{1}{2m}\sum(a_3 - y)^2$$

The purpose of the loss function is to find the optimal values of the weights such that J is minimized since the corresponding weights provide the most accurate predictions for a_3. To find those weights, that is where the concept of calculus is applied to the back propagation process.

Back Propagation:
To obtain a good model, the weights must be adjusted such that the loss function is minimized. Any changes to the weights directly affect the loss function hence the partial derivative of loss with respect to the weights come to interest. It is observed that if the node using a given weight is relatively large, then manually adjusting the weight would give a corresponding change to the loss, then the derivative must also depend on the values of the nodes, which are the elements of our matrices.

As it turns out, we can use chain rule so that we can determine $\frac{\partial J}{\partial \theta}$.

$$\frac{\partial J}{\partial \theta_1} = \frac{\partial J}{\partial a_3}\frac{\partial a_3}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial \theta_1}$$

$$\frac{\partial J}{\partial \theta_2} = \frac{\partial J}{\partial a_3}\frac{\partial a_3}{\partial z_3}\frac{\partial z_3}{\partial \theta_2}$$

The different derivatives are evaluated as:

$$\frac{\partial J}{\partial a_3} = \frac{1}{m}(a_3 - y)$$

$$\frac{\partial a_3}{\partial z_3} = \sigma'(z_3) = \sigma(z_3)[1 - \sigma(z_3)]$$

$$\frac{\partial z_3}{\partial a_2} = \theta_2$$

$$\frac{\partial a_2}{\partial z_2} = \sigma'(z_2) = \sigma(z_2)[1 - \sigma(z_2)]$$

$$\frac{\partial z_2}{\partial \theta_1} = a_1$$

$$\frac{\partial z_3}{\partial \theta_2} = a_2$$

Therefore,

$$\frac{\partial J}{\partial \theta_1} = \left(\left\{\left[\frac{1}{m}(a_3 - y) * \sigma'(z_3)\right]\theta_2\right\} * \sigma'(z_2)\right)^T a_1$$

$$\frac{\partial J}{\partial \theta_2} = \left[\frac{1}{m}(a_3 - y) * \sigma'(z_3)\right]^T a_2$$

Note here that $*$ denotes elementwise multiplication since those matrices have equal sizes. It is also because the multiplication by the gradient of the activation function means a rescaling of the error. There are also transpositions that occurred such that matrix multiplication is applicable. Also note that after the matrix multiplication with the weights is performed, do remember to remove the appended column since this is the bias term, and the bias term are not propagated back to previous layers.

Once these partial derivatives are calculated, it is then multiplied to some carefully selected learning rate α, then subtracted to the weights prior to back propagation.

Training:
Training is the process of performing multiple iterations of forward propagation, loss computing, and back propagation. Training basically means readjusting the weights to such a point that the next forward propagation provides a low value for loss. For initialization of weights, a seed corresponding to the answer to life, the universe, and everything (42) is selected for reproducibility of results.

Here are the following parameters used during training:

Epochs	100
α	0.0005
Weights initialization	$U(0,1) - 0.5$ *
Activation Function, $\sigma(z)$	$\frac{1}{1+e^{-z}}$
Input Layer Size	401**
Hidden Layer Size (1 layer only)	26**
Output Layer Size	10
Loss Function, J	$\frac{1}{2m}\sum(y_{pred} - y)^2$

Notes:
*Uniform Distribution between 0—1 then centered around 0.
**Includes bias term hence the +1.

With these parameters, plot of loss vs number of epochs. Classification error is also shown.

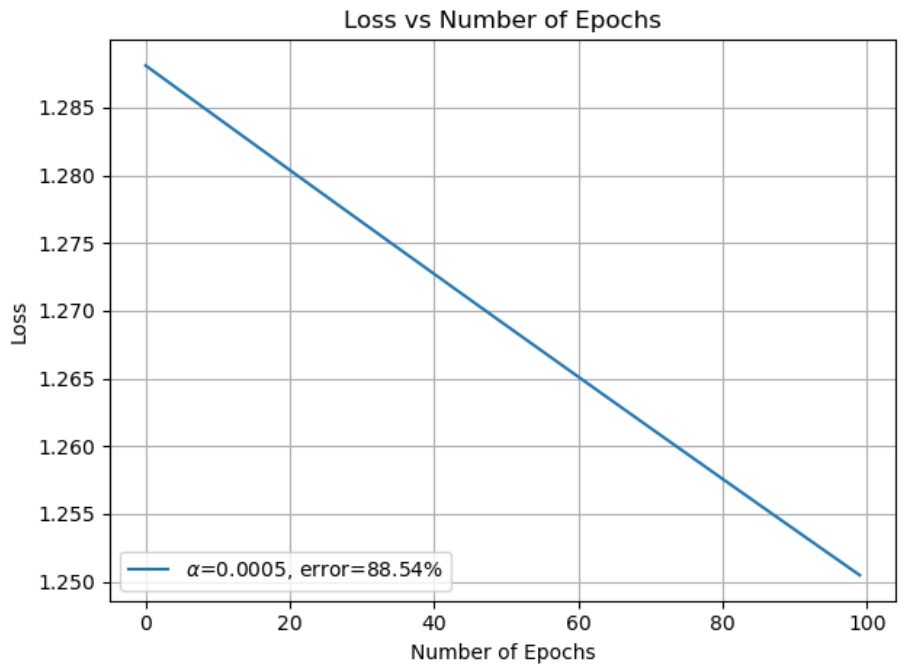


Figure 8: A plot for a small learning rate of 0.0005 and trained for 100 epochs. The loss value decreases linearly however it shows 88.54% classification error, even to the data it is trained on.

With nothing to compare, the loss seems to decrease linearly as epochs increase. To verify this, let us try to use multiple learning rates α and increase epochs to 500.

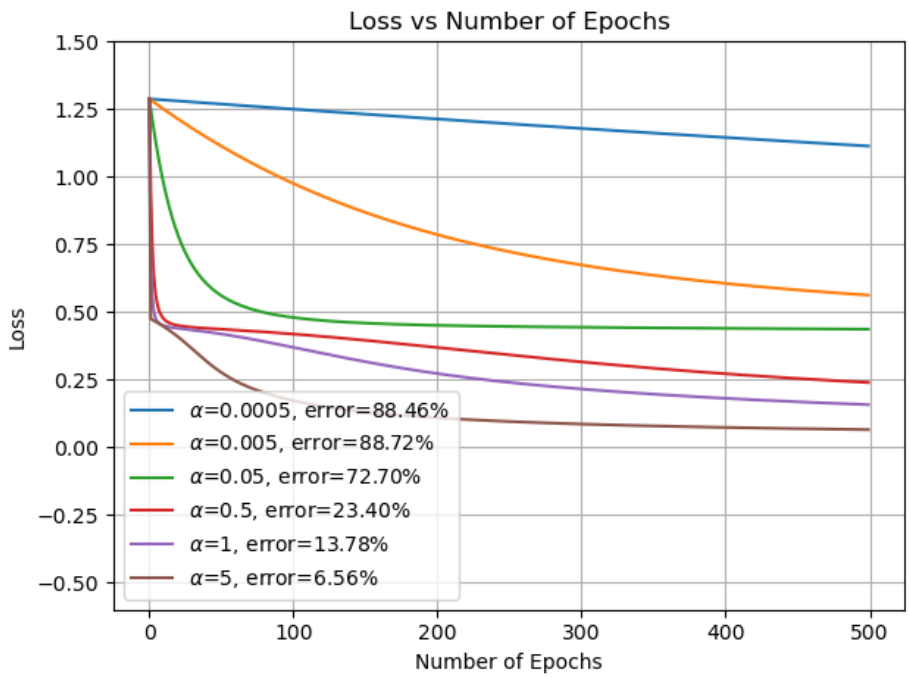


Figure 9: A comparison of loss vs epochs but for the same ANN trained on 500 epochs but each differing only on the learning rate used. As observed, the loss decrease of the first model is almost negligible when compared to the others. Also, the classification error becomes significantly lower which is now able to predict much more correctly.

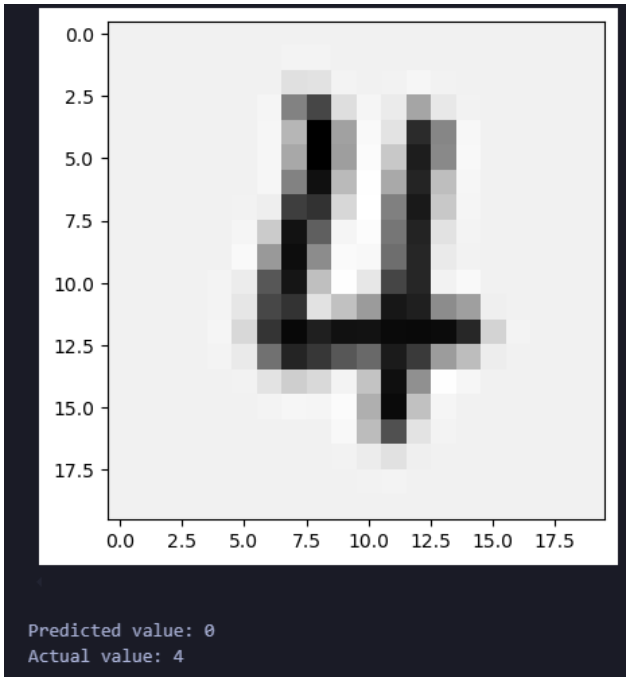


Figure 10: For the model with learning rate of 0.0005 and trained at 100 epochs.

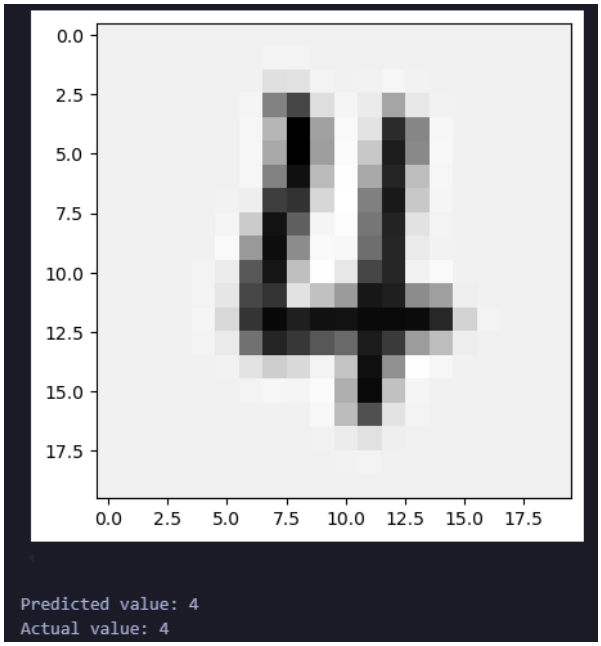


Figure 11: For the model with learning rate of 5 and trained at 500 epochs.

4. Hybrid Optimization

First, the Broyden-Fletcher-Goldfarb-Shanno (BFGS) Algorithm was attempted but with a constant learning rate. BFGS is an attractive algorithm to use as the base as it does not require to perform higher level differentiation and does not require the calculation of an inverse of a matrix.

$$x^{(n+1)} = x^{(n)} - \alpha B^{(n)} \nabla f\left(x^{(n)}\right)$$

$$B^{(n+1)} = B^{(n)} + \frac{y^{(n)} y^{(n)T}}{y^{(n)T} s^{(n)}} - \frac{B^{(n)} s^{(n)} s^{(n)T} B^{(n)T}}{s^{(n)T} B^{(n)} s^{(n)}}$$

Where:

$n \rightarrow$ denotes the number of iterations

$B \rightarrow$ approximation of the inverse of Hessian

$s^{(n)} = x^{(n+1)} - x^{(n)} \rightarrow$ step taken

$y^{(n)} = \nabla f\left(x^{(n+1)}\right) - \nabla f\left(x^{(n)}\right) \rightarrow$ change in gradient

However, the issue encountered when performing a pure BFGS, the calculated function $f(x, y)$ becomes unstable and the values overshoot to infinity. If the chosen learning rate is a very small number to avoid overshooting, the algorithm takes a very long time and still does not approach convergence. The idea of having a dynamic learning rate or step size is needed such that we can have a large step size at the beginning and gradually decrease step size when near a minimum since having a very large step size would risk overshooting.

With this, backtracking line search using Armijo's rule is used. Armijo's rule requires a condition such that when new function value after the step is two large that would risk overshooting, it dials down the step size by a given factor.

$$f(x) - f(x + \alpha p) \geq -\alpha c \nabla f(x)^T p$$

Where:

$p \rightarrow$ The chosen direction of descent. When combined with BFGS, $p = -B\nabla f(x)$.

$c \rightarrow$ A control parameter $c \in (0, 1)$. A popular value would be 0.0001.

The step size α is usually decreased by the factor $\tau \in (0, 1)$ with popular values ranging from 0.5-0.9.

For this method, convergence is achieved by determining the direction of descent based on the linear transformation of the Broyden to the gradient of the function. To find a good amount of step size, a relatively large step size is first chosen. If the step is believed to be too large, based on Armijo's rule, the step size is dialed down until such a point that the step size is enough. While this involves another iteration within an iteration, this is much cheaper than the Newton's method but at the same time we optimize the step size. The problem with a constant step size is if it is too large, we risk overshooting. If it is too small, the iterations take too long.

After some changes to α, the algorithm was applied to the test with the Beale function.

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$$

The global minima for this function occurs at $f(3.0, 0.5) = 0$.

This is a good function to test as this has a lot of local minima. One of the challenges was that if the independent variables were initialized randomly, they would converge to some local minima that is not the global minima, despite increasing the iterations.

Another method used together with the BFGS is the trust region method. Here, the objective function is approximated to be a quadratic function

$$f_{approx}(x) = f(x) + \nabla f(x)^T p + \frac{1}{2} p^T B(x) p$$

The trust region radius Δ is selected such that the function is trusted to approximate well within the said radius. If Δ is large, initial p would rescale to have the same length as Δ. The p is then used to calculate a condition parameter ρ.

$$\rho = \frac{f\left(x^{(n)}\right) - f\left(x^{(n)} + p\right)}{f(x^{(n)}) - f_{approx}(x^{(n)})} = -\frac{f\left(x^{(n)}\right) - f\left(x^{(n)} + p\right)}{\nabla f(x^{(n)})^T p + \frac{1}{2} p^T B(x^{(n)}) p}$$

$\rho \rightarrow$ This is a measure of the ratio of actual reduction and prediction reduction. If it is less than 0.25, reduce the trust region radius 0.25 of the original. If it is greater than 0.75, choose the minimum between the original selected Δ or twice the new Δ.

BFGS is then performed using the approximated function within the trust radius. Instead of estimating a step size based on some constant, the step size is determined from the trust radius. It is dynamically adjusted per iteration based on the ρ. With this, we can converge to a minimum.

The computational cost here when compared to other methods such as the Newton's method (Newton's method is considered the benchmark) is cheaper as it calculates the step size based on matrix operations and does not include any iteration within the actual descent iteration.

Unfortunately, the same issue as BFGS with backtrack line search is encountered – it gets stuck on a local minimum. The work around for both hybrid models is to perform multiple initial start points then compare the convergence point of all those instances. The instance with the lowest objective function is then selected.

Shown are the direction of descent of the instance that approached the global minimum. Both algorithms were set to a maximum iteration of 500 (but iterations can be cut off early if values converged to a tolerance of 1e-5). Although not exactly $f(3.0, 0.5) = 0$, due to the approximation involved of BFGS, this tradeoff is considered acceptable as the error is not that big.

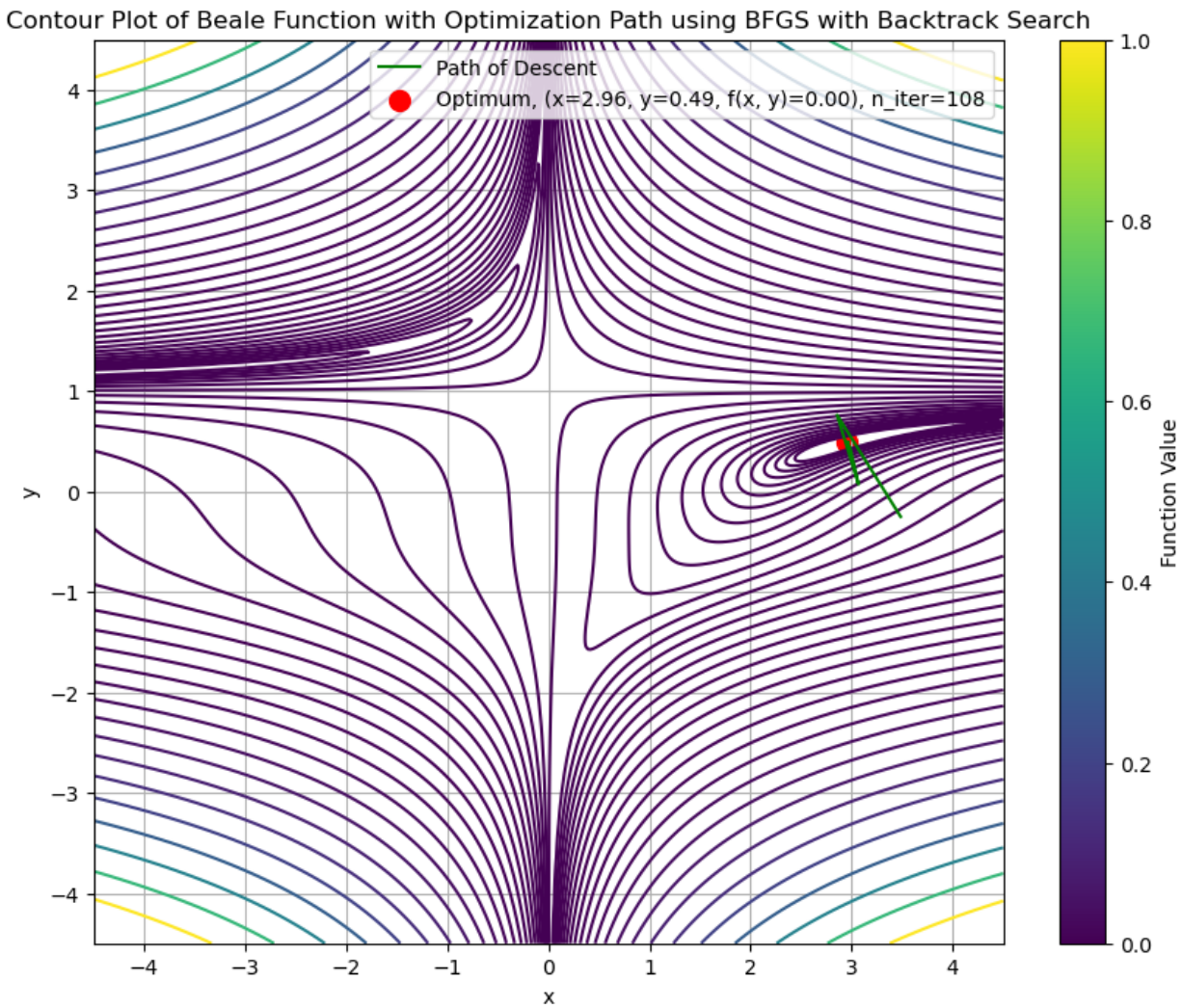


Figure 12: The trajectory of descent of the BFGS with Backtrack search model. The model was randomly initialized to 50 starting points and the point with the lowest function value is shown. This was the solution thought to solve the model getting stuck at the local minima.

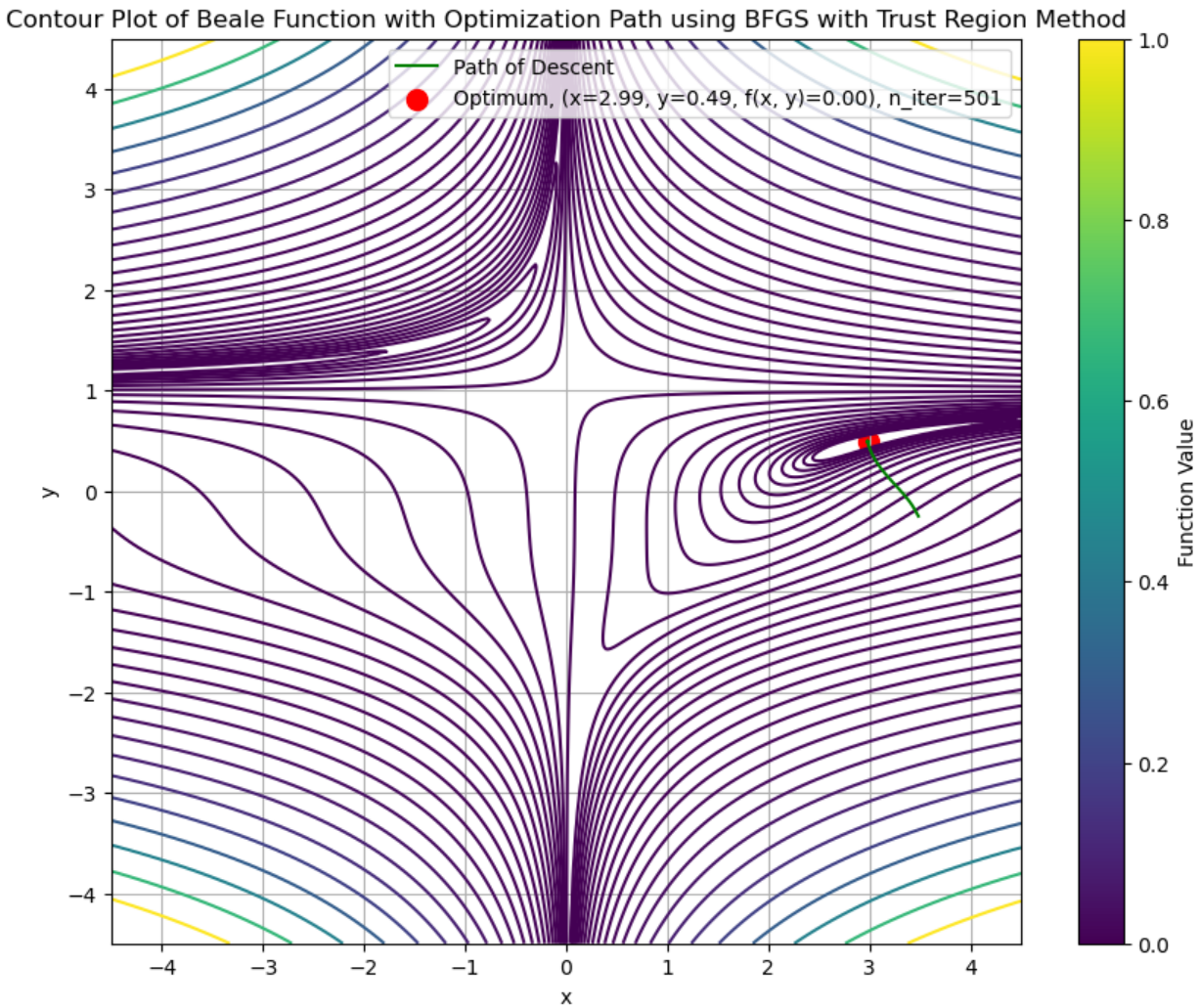


Figure 13: The trajectory of descent of the BFGS with Trust Region method model. The model was randomly initialized to 50 starting points and the point with the lowest function value is shown. This was the solution thought to solve the model getting stuck at the local minima.

Appendix

Eigenfaces and Face Recognition

Importing the necessary libraries. Note that the actual logic and algorithms are implemented purely on numpy. The imported "ai211" is a custom python file to store necessary functions and classes.

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat

from ai211 import svd, inverse, scale_data, SimpleANN, norm2
```

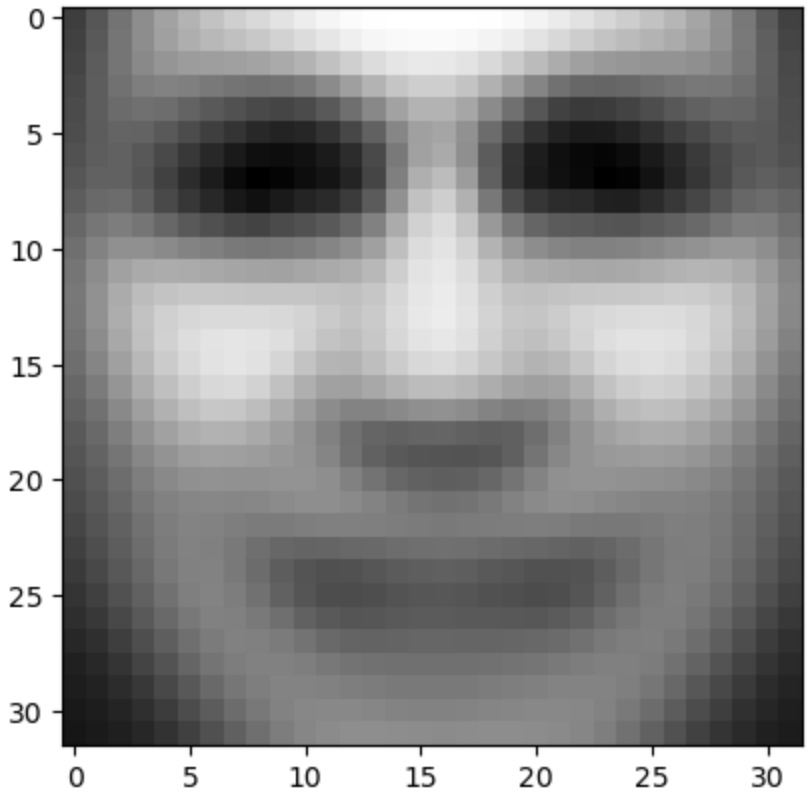
Define a function that turns each row into a 2 dimensional matrix that can be viewed as an image.

In [ ]:
```
def plot_faces(X, dim=(10, 10)):
    m = int(dim[0])
    n = int(dim[1])
    fig, axes = plt.subplots(m, n, figsize=(10, 10))
    for idx, ax in enumerate(axes.flat):
        ax.imshow(X[idx].reshape(32, 32).T, cmap='gray')
        ax.axis('off')
```

Load the faces into a matrix. Here, the mean of the faces is subtracted so that the data only contains the important variances that define the faces. The average or mean face is also visualized.

In [ ]:
```
X = loadmat('faces.mat')['X']
X_mean = X.mean(axis=0)
X_no_mean = X - X_mean
```

In [ ]:
```
plt.imshow(X_mean.reshape(32, 32).T, cmap='gray');
```



For the next line, it is shown here the decomposition of the dataset. The actual implementation of the function can be found in the ai211 python file. Also, the decomposed matrices are saved to a .npz file since the decomposition took some time.

In [ ]:
```
# Takes some time to run. Saved the arrays to 'USVt_arr.npz' instead.
U, S, Vt = svd(X_no_mean)
```

In [ ]:
```
np.savez('USVt_arr', U=U, S=S, Vt=Vt)
```

When the kernel of this notebook is restarted, the arrays are saved on the external file.

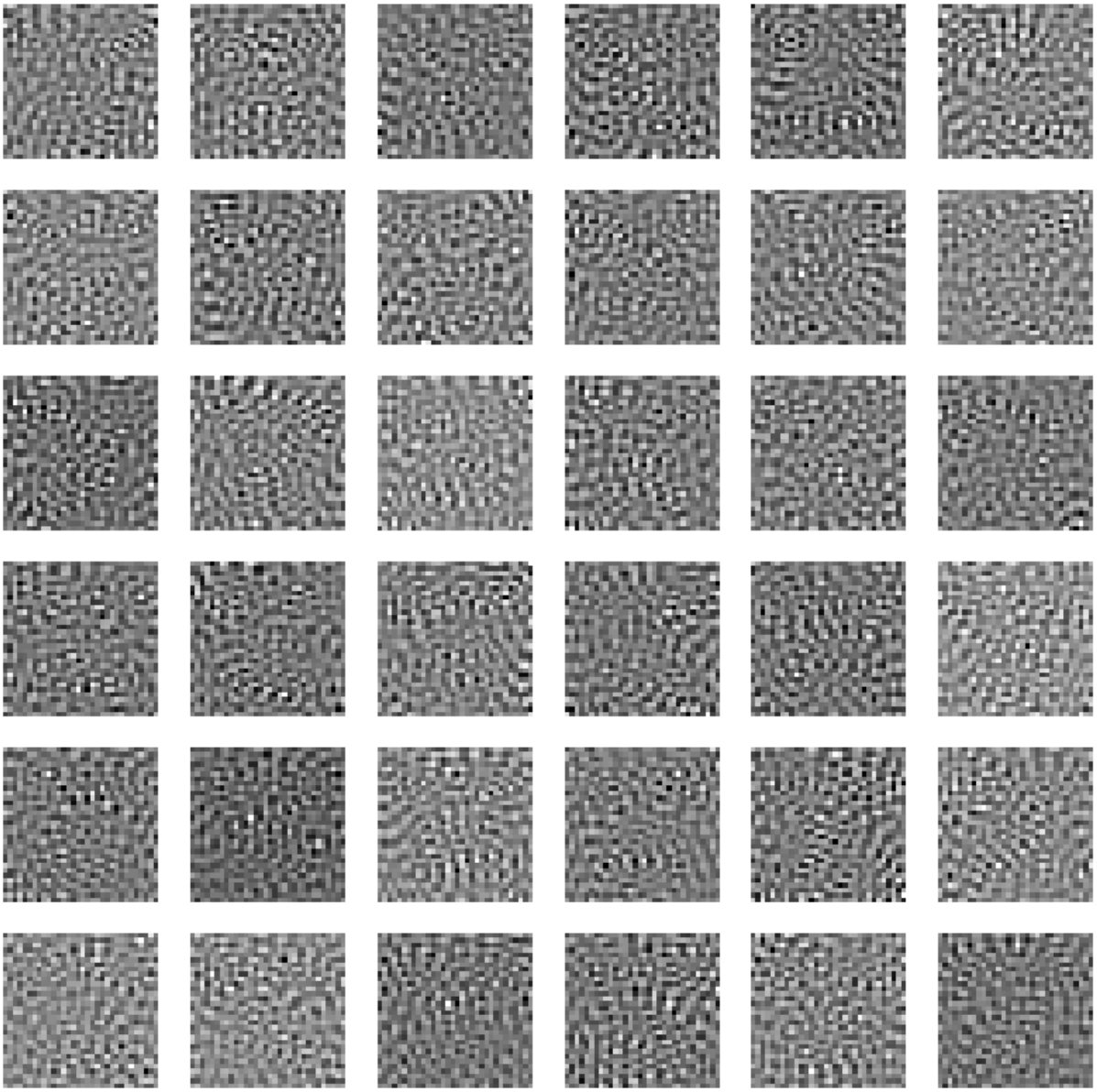
In [ ]:
```
decomposed_arr = np.load('USVt_arr.npz')
U = decomposed_arr['U']
S = decomposed_arr['S']
Vt = decomposed_arr['Vt']
```

The matrix Vt containing the eigenfaces are plotted. Shown here are the first 36 eigenfaces. These show the important features that make up the faces.

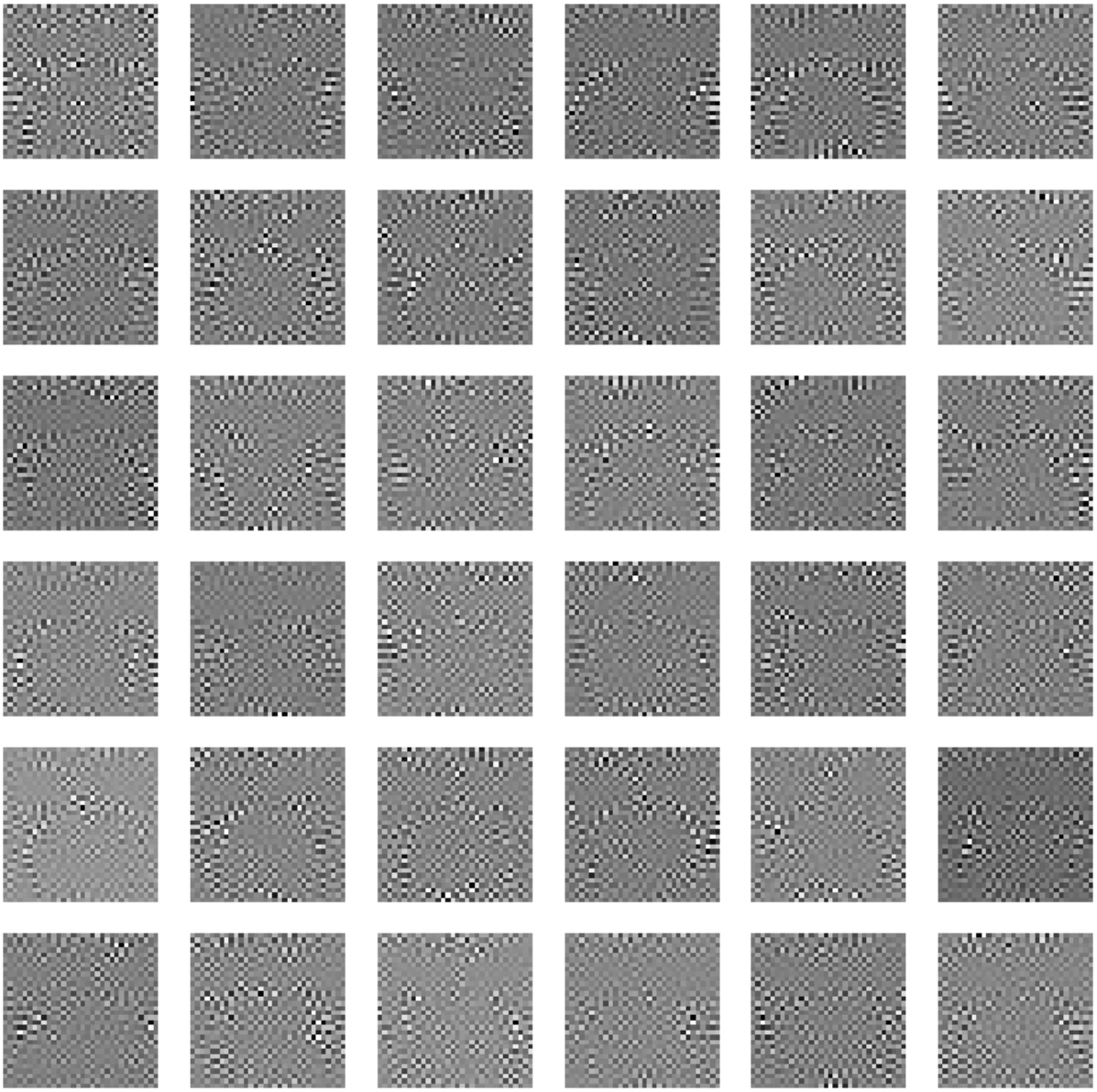
In [ ]:
```
plot_faces(-Vt, (6, 6))
```

The next two plots also show eigenfaces but corresponds to a low singular value. The plots are observed to look like noise and contains little information.

In [ ]:
```
plot_faces(-Vt[400:], (6, 6))
```



In [ ]:
```
plot_faces(-Vt[900:], (6, 6))
```

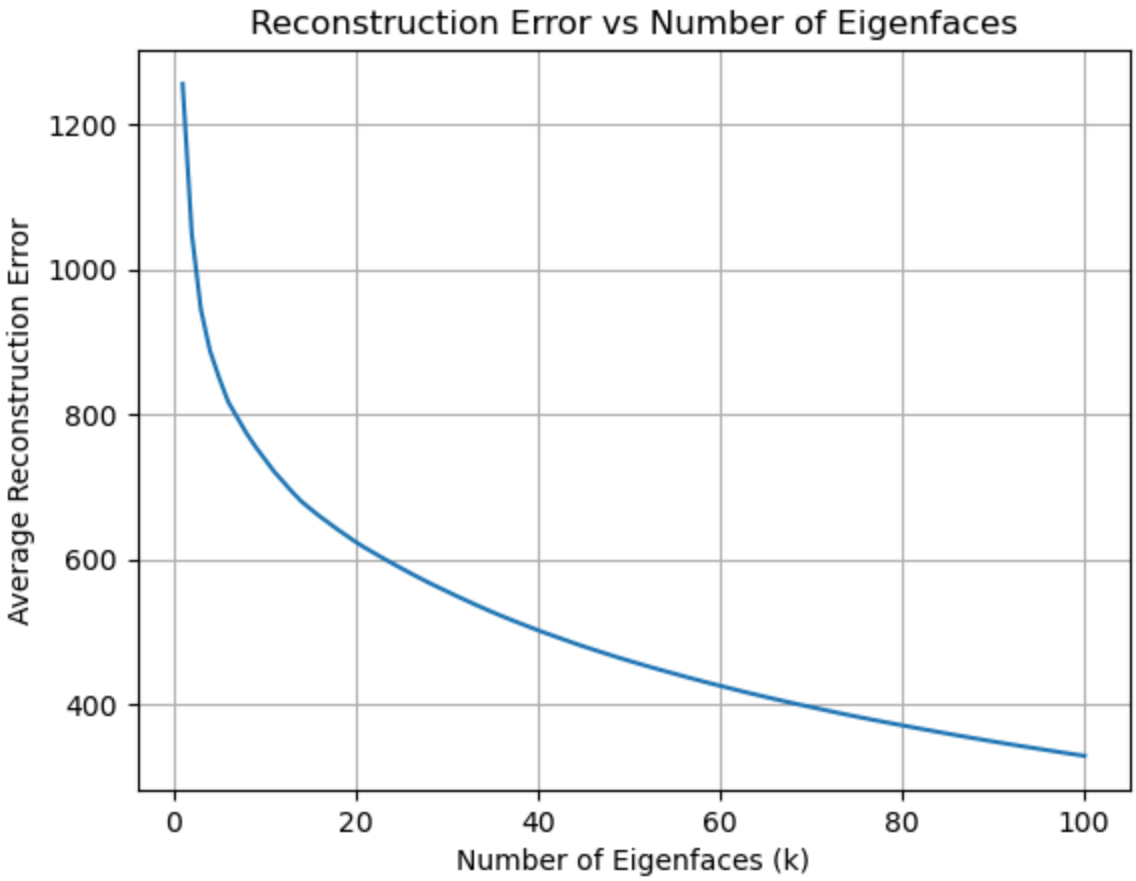


Shown is the plot of reconstruction error vs eigenfaces. The reconstruction error at the early eigenfaces greatly decreases however the decrease start to decrease.

In [ ]:
```
n = 100
errors = []
for k in range(0, n):
    U_k = U[:, :k]
    S_k = S[:k, :k]
    Vt_k = Vt[:k, :]
    X_reconstructed = U_k @ S_k @ Vt_k + X_mean
    error = X - X_reconstructed
    error = np.mean(np.linalg.norm(error, axis=1), axis=0)
    errors.append(error)
```

In [ ]:
```
plt.plot(range(1, n+1), errors)
plt.xlabel('Number of Eigenfaces (k)')
plt.ylabel('Average Reconstruction Error')
```

```
plt.title('Reconstruction Error vs Number of Eigenfaces')
plt.grid()
```



For the first 100 eigenfaces, the reconstructed pictures does not look that bad and very similar to the first with some few information losses. This proves that only the eigenfaces corresponding to high singular values carry the most information while the remaining contains too much randomness or noise.

In [ ]:
```
U_100 = U[:, :100]
S_100 = S[:100, :100]
Vt_100 = Vt[:100, :]
X_reconstructed = U_100 @ S_100 @ Vt_100 + X_mean
plot_faces(X_reconstructed, (10, 10))
```

Nonlinear vs Linearized Least Squares Regression

Defining the experimental or measured values. Also the necessary functions are hardcoded for this case.

In [ ]:
```
C = np.array([2.5, 5., 10., 15., 20.])
v0 = np.array([0.024, 0.036, 0.053, 0.06, 0.064])
```

In [ ]:
```
def v0_model(C, Vmax, Km):
    return Vmax / (1 + Km/C)

def residuals(theta, C, v0):
    return v0_model(C, theta[0], theta[1]) - v0

def J_r(theta, C):
    Vmax, Km = theta
    J = np.zeros((len(C), len(theta)))
    J[:, 0] = C / (C + Km)
```

```
    J[:, 1] = -(C * Vmax) / (C + Km) ** 2
    return J
```

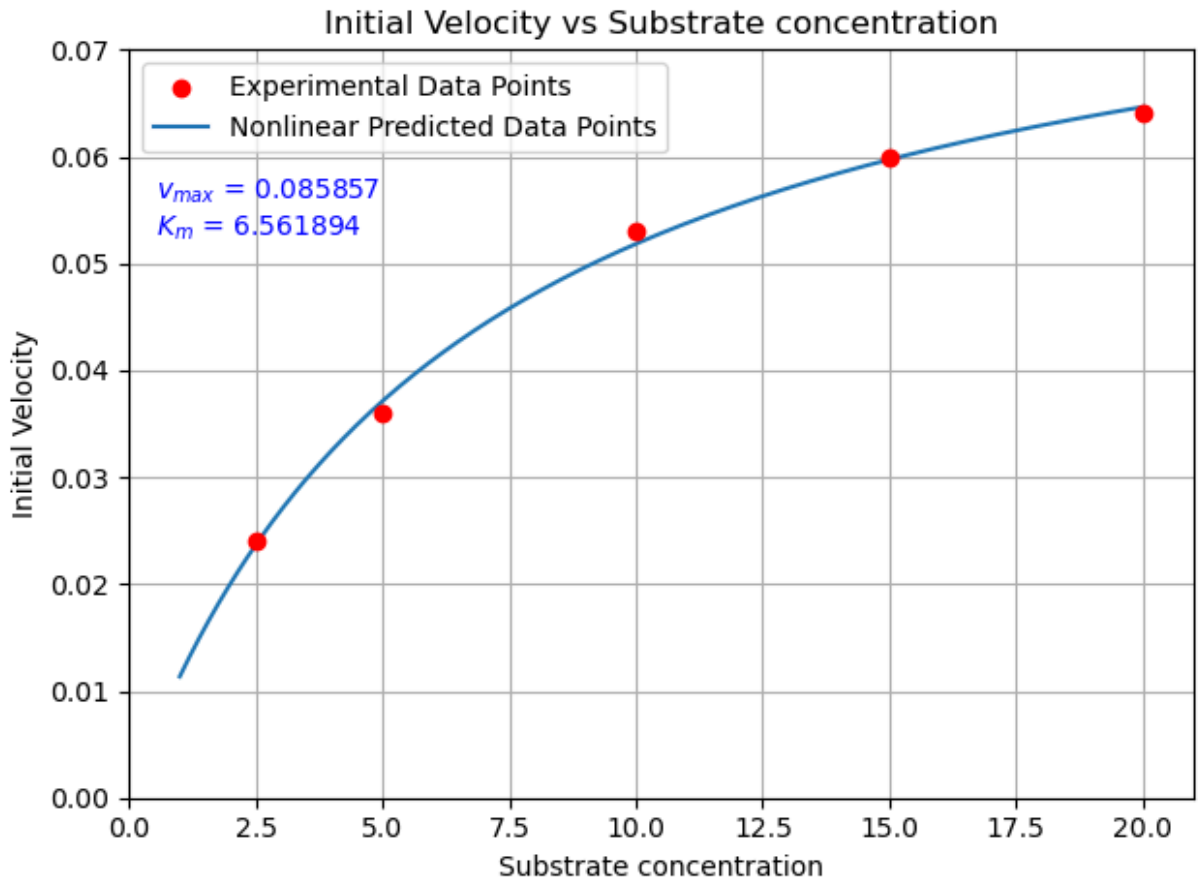
Initializing the parameters and then performing Gauss-Newton to solve for the parameters.

In [ ]:
```
theta = np.random.randint(size=(2,), low=1, high=2)
# Iteration
for _ in range(100):
    r = residuals(theta, C, v0)
    J = J_r(theta, C)
    H = J.T @ J
    theta = theta - inverse(H) @ J.T @ r

print(f'Vmax = {theta[0]:.6f}; Km = {theta[1]:.6f}')
```

```
Vmax = 0.085857; Km = 6.561894
```

Plot for experimental data vs predicted data is shown.

In [ ]:
```
vmax_text = f'$v_{{max}}$ = {theta[0]:.6f}'
km_text = f'$K_m$ = {theta[1]:.6f}'

C_fit = np.linspace(1, 20, 400)
nl_v0_fit = v0_model(C_fit, theta[0], theta[1])

plt.scatter(C, v0, color='r', label='Experimental Data Points', zorder=3)
plt.plot(C_fit, nl_v0_fit, label='Nonlinear Predicted Data Points', zorder=2)
plt.grid(True)
plt.xlabel('Substrate concentration')
plt.ylabel('Initial Velocity')
plt.title('Initial Velocity vs Substrate concentration');
plt.legend()
plt.xlim(0, 21)
plt.ylim(0, 0.07)
plt.xticks(np.arange(0, 21, 2.5))
plt.yticks(np.arange(0, 0.071, 0.01))
plt.text(0.025, 0.8, vmax_text, transform=plt.gca().transAxes, fontsize=10,
         color='blue')
plt.text(0.025, 0.75, km_text, transform=plt.gca().transAxes, fontsize=10,
         color='blue')
plt.tight_layout()
```



A direct solution for solving linear models.

In [ ]:
```
def solve_theta(cols, y):
    A = np.column_stack(cols)
    U, S, Vt = svd(A)
    S_pinv = inverse(S.T @ S) @ S.T
    theta = Vt.T @ S_pinv @ U.T @ y

    return theta
```

Solving the 3 linear models.

In [ ]:
```
# Lineweaver
lineweaver_y = 1 / v0
lineweaver_col_0 = np.ones(5)
lineweaver_col_1 = np.array([1/c for c in C])

lineweaver_theta = solve_theta(
    (lineweaver_col_0, lineweaver_col_1),
    lineweaver_y
)

lineweaver_vmax = 1 / lineweaver_theta[0]
lineweaver_Km = lineweaver_theta[1] * lineweaver_vmax
print(f'Lineaweaver vmax = {lineweaver_vmax:.6f}')
print(f'Lineaweaver Km = {lineweaver_Km:.6f}')
```

```
Convergence
Lineaweaver vmax = 0.084743
Lineaweaver Km = 6.392313
```

In [ ]:
```
# Dixon
dixon_y = C / v0
dixon_col_0 = np.ones(5)
dixon_col_1 = C

dixon_theta = solve_theta(
    (dixon_col_0, dixon_col_1),
    dixon_y
)

dixon_vmax = 1 / dixon_theta[1]
dixon_Km = dixon_theta[0] * dixon_vmax
print(f'Dixon vmax = {dixon_vmax:.6f}')
print(f'Dixon Km = {dixon_Km:.6f}')
```

```
Convergence
Dixon vmax = 0.085339
Dixon Km = 6.469373
```

In [ ]:
```
# Eadie
eadie_y = v0
eadie_col_0 = np.ones(5)
eadie_col_1 = -v0 / C

eadie_theta = solve_theta(
    (eadie_col_0, eadie_col_1),
    eadie_y
)

eadie_vmax, eadie_Km = eadie_theta
print(f'Eadie vmax = {eadie_vmax:.6f}')
print(f'Eadie Km = {eadie_Km:.6f}')
```

```
Convergence
Eadie vmax = 0.085581
Eadie Km = 6.515470
```

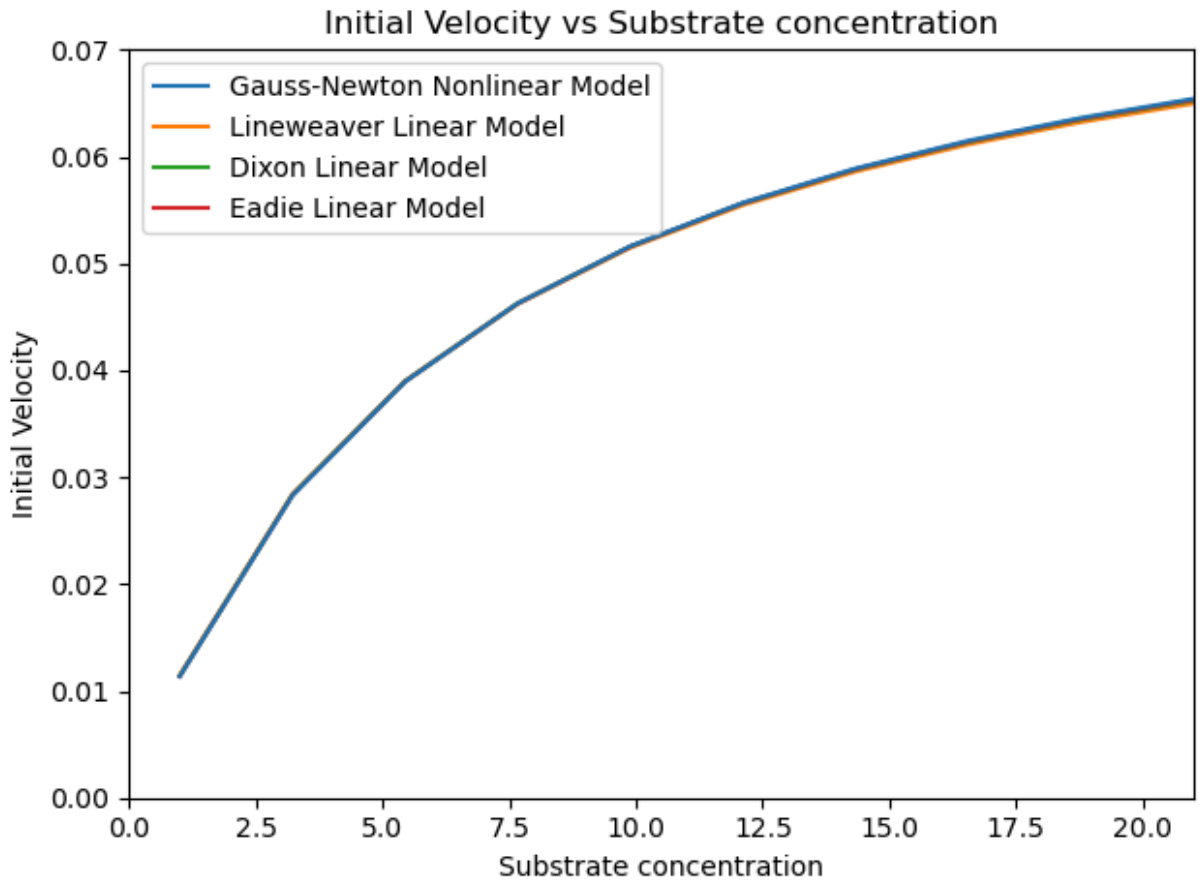
Showing the errors when compared to the gauss newton method.

In [ ]:
```
print('Errors compared to vmax')
for i in [theta[0], lineweaver_vmax, dixon_vmax, eadie_vmax]:
    print(f'{abs((theta[0] - i)/theta[0])*100:.2f}')
print('')

print('Errors compared to Km')
for i in [theta[1], lineweaver_Km, dixon_Km, eadie_Km]:
    print(f'{abs((theta[1] - i)/theta[1])*100:.2f}')
print('')
```

```
Errors compared to vmax
0.00
1.30
0.60
0.32

Errors compared to Km
0.00
2.58
1.41
0.71
```

Showing the plots of the 4 models (1 NL and 3 Linear). Viusually, very little error can be observed.

In [ ]:
```
C_fit_2 = np.linspace(1, 21, 10)
nl_v0_fit = v0_model(C_fit_2, theta[0], theta[1])
lineweaver_v0_fit = v0_model(C_fit_2, lineweaver_vmax, lineweaver_Km)
dixon_v0_fit = v0_model(C_fit_2, dixon_vmax, dixon_Km)
eadie_v0_fit = v0_model(C_fit_2, eadie_vmax, eadie_Km)

plt.plot(C_fit_2, nl_v0_fit, label='Gauss-Newton Nonlinear Model', zorder=4)
plt.plot(C_fit_2, lineweaver_v0_fit, label='Lineweaver Linear Model')
plt.plot(C_fit_2, dixon_v0_fit, label='Dixon Linear Model')
plt.plot(C_fit_2, eadie_v0_fit, label='Eadie Linear Model')
plt.xlabel('Substrate concentration')
plt.ylabel('Initial Velocity')
plt.title('Initial Velocity vs Substrate concentration');
plt.legend()
plt.xlim(0, 21)
plt.ylim(0, 0.07)
plt.xticks(np.arange(0, 21, 2.5))
plt.yticks(np.arange(0, 0.071, 0.01))
plt.tight_layout()
```



MNIST Handwritten Digit Classification Using ANN

Loading the features X and label y. The 10's in the y are reencoded to 0 for convenience.

In [ ]:
```
data = loadmat('mnist.mat')
X = data['X']
y = data['y']
y = np.where(y==10, 0, y)
```

The class SimpleANN and methods can be seen on ai211. The dataset is initially trained using suggested parameters.

In [ ]:
```
model = SimpleANN(
    input_layer_size=400,
    hidden_layer_size=25,
    num_labels=10,
    alpha=0.0005,
    n_iters=100
)
model.train(X, y)
```

Defining some functions.

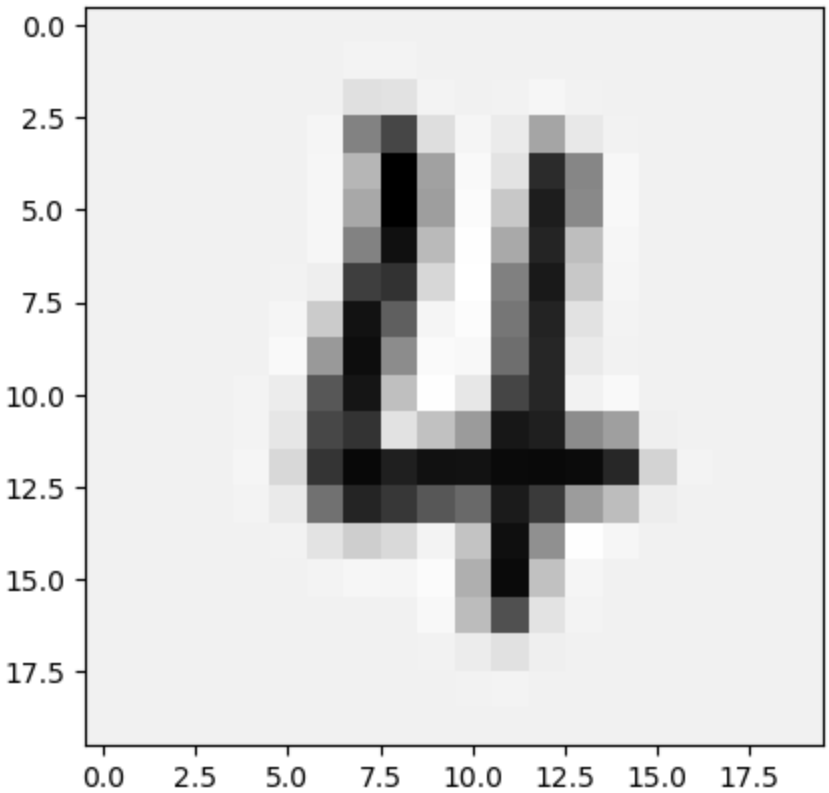
In [ ]:
```
def show_incorrect_preds(X, y, model):
    predictions = model.predict(X)
    return np.argwhere(np.where(predictions!=y.flatten(), 1, 0)).reshape(-1)
```

```
def show_classifcation_error(X, y, model):
    predictions = model.predict(X)
    error_count = np.argwhere(
        np.where(predictions!=y.flatten(), 1, 0)
    ).shape[0]
    return error_count / X.shape[0] * 100

def show_predict(X, pic_pos, model):
    predictions = model.predict(X)
    plt.imshow(-X[pic_pos].reshape(20, 20).T, cmap='gray')
    plt.show()
    print(f'Predicted value: {predictions[pic_pos]}')
    print(f'Actual value: {y.flatten()[pic_pos]}')
```

In [ ]:
```
def plot_loss_epoch_per_alpha(alphas=[0.00005], n_iter=100):
    X_y_pairs = []
    clf_errors = []
    for alpha in alphas:
        model = SimpleANN(
            input_layer_size=400,
            hidden_layer_size=25,
            num_labels=10,
            alpha=alpha,
            n_iters=n_iter
        )
        model.train(X, y)
        X_alpha = np.arange(model.epochs)
        y_alpha = model.losses
        X_y_pairs.append((X_alpha, y_alpha))
        clf_errors.append(show_classifcation_error(X, y, model))

    for i in range(len(alphas)):
        plt.title('Loss vs Number of Epochs')
        plt.plot(
            X_y_pairs[i][0],
            X_y_pairs[i][1],
            label=rf'$\alpha$={alphas[i]}, error={clf_errors[i]:.2f}%'
        )
    plt.title('Loss vs Number of Epochs')
    plt.xlabel('Number of Epochs')
    plt.ylabel('Loss')
    if np.min(X_y_pairs[-1][1]) < 1.15:
        plt.ylim(-.6, 1.5)
    plt.legend(loc='lower left')
    plt.tight_layout()
    plt.grid(True)
```

As seen, model predicts poorly. Classification error can be seen on next plots.

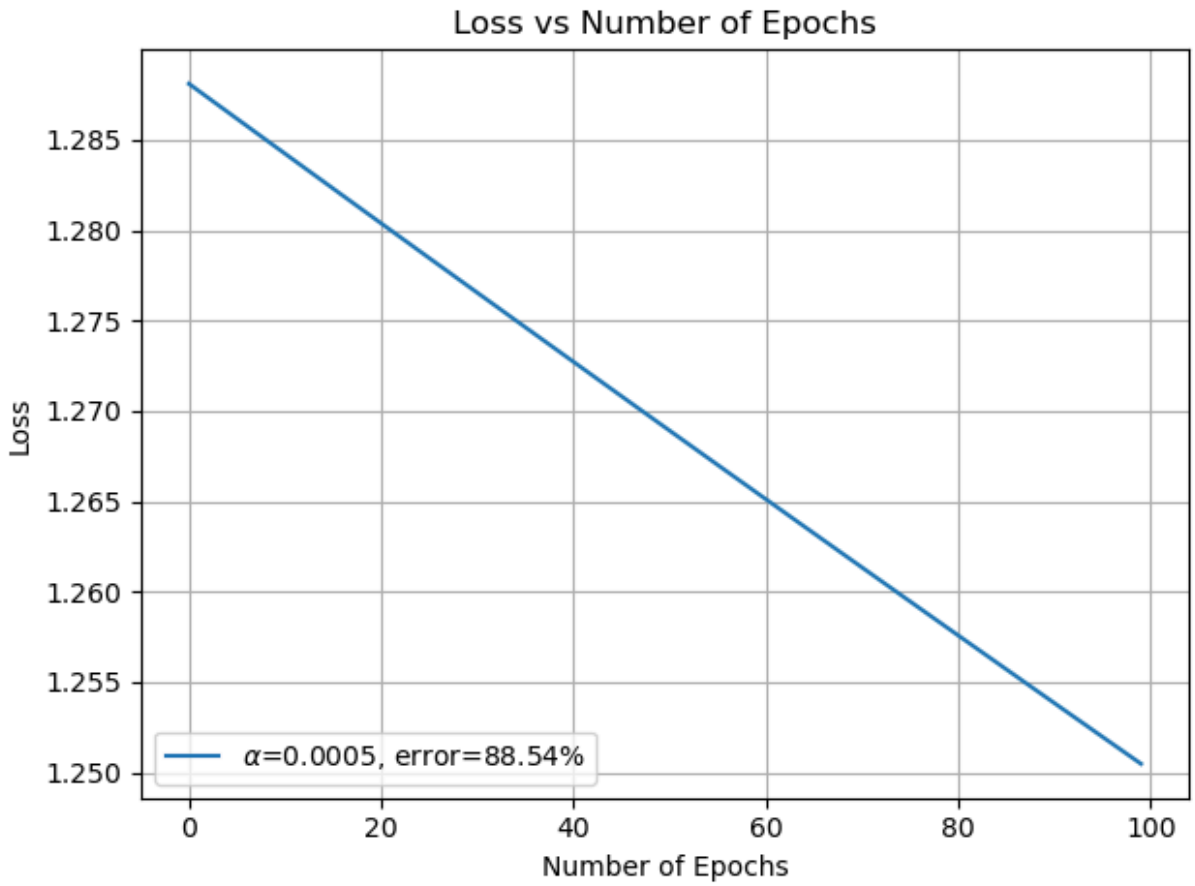
In [ ]:
```
show_predict(X, 2324, model)
```




```
Predicted value: 0
Actual value: 4
```

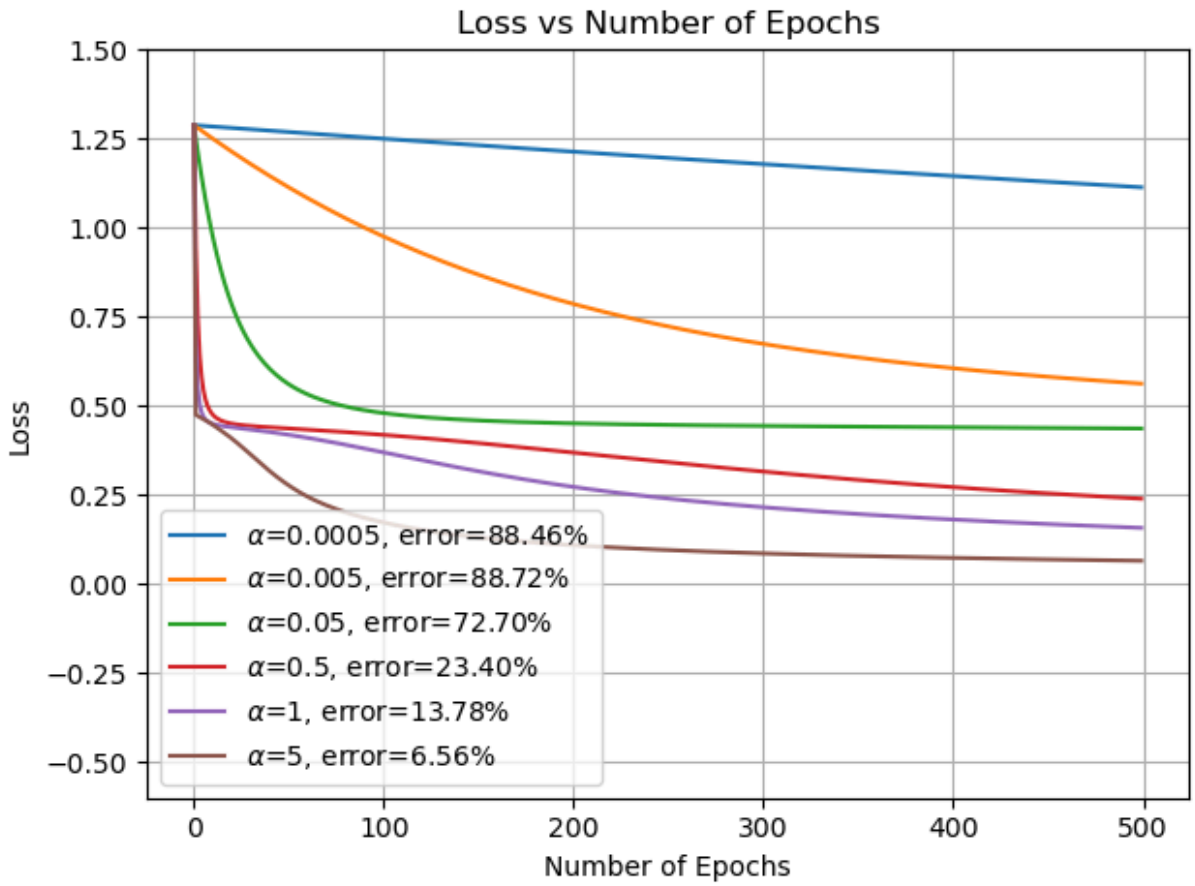
Plotting the loss vs epoch for the initial alpha and 100 iterations (the first model that was trained which predicted poorly). Model looked like it is decreasing linearly however it is unknown to us if this is a good decrease of loss. Let us try to compare with other alpha or learning rate and also on a higher number of iterations. However, note that it yields a very high classification error. This meant that out of the 5000 images, it was only able to correctly predict around 600 images.

In [ ]:
```
plot_loss_epoch_per_alpha(alphas=[0.0005], n_iter=100)
```



Here, it is observed that the loss decrease for the very first model was really negligible. For the other trained models at a higher learning rate, the decrease in loss is much more noticeable and yields a much lower classification error. However, the risk of overfitting could be encountered, but that is not the point for now.

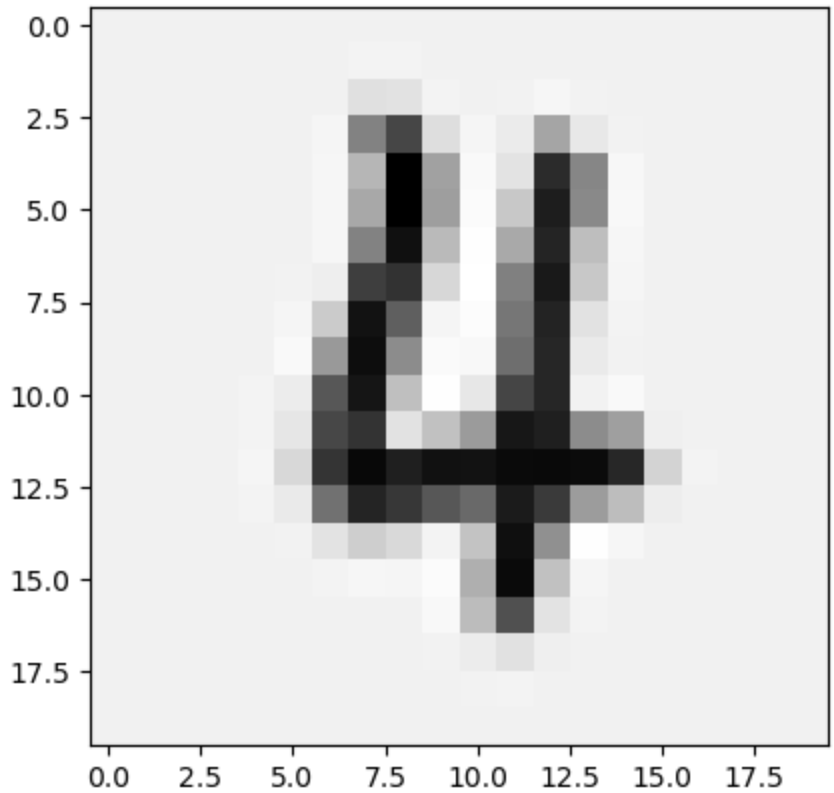
In [ ]:
```
alphas = [0.0005, 0.005, 0.05, 0.5, 1, 5]
plot_loss_epoch_per_alpha(alphas, 500)
```



Using the model with a higher alpha and higher iterations, we can see that it was now able to predict the image that was icorrectly predicted by our initial model.

In [ ]:
```
model = SimpleANN(
    input_layer_size=400,
    hidden_layer_size=25,
    num_labels=10,
    alpha=5,
    n_iters=500
)
model.train(X, y)
```

In [ ]:
```
show_predict(X, 2324, model)
```




```
Predicted value: 4
Actual value: 4
```

Hybrid Optimization

Hardcoding the Beale function and its gradient.

In [ ]:
```
def obj_func(X):
    x1, x2 = X[0], X[1]
    z = (1.5 - x1 + x1 * x2)**2 + \
        (2.25 - x1 + x1 * x2**2)**2 + \
        (2.625 - x1 + x1 * x2**3)**2
    return z


def obj_func_grad(X):
    x1, x2 = X[0], X[1]
    df_dx1 = 2 * (1.5 - x1 + x1 * x2) * (-1 + x2) + \
             2 * (2.25 - x1 + x1 * x2**2) * (-1 + x2**2) + \
             2 * (2.625 - x1 + x1 * x2**3) * (-1 + x2**3)
    df_dx2 = 2 * (1.5 - x1 + x1 * x2) * x1 + \
             4 * (2.25 - x1 + x1 * x2**2) * x1 * x2 + \
             6 * (2.625 - x1 + x1 * x2**3) * x1 * x2**2
    return np.array([df_dx1, df_dx2])
```

Defining BFGS with backtrack search and BFGS with trust region method.

In [ ]:
```
def line_search_armijo(X, p, obj_func, obj_func_grad,
                       alpha=1,
                       tau=0.7,
```

```
                        c=1e-4):
    m = obj_func_grad(X).T @ p
    while obj_func(X) - obj_func(X + alpha * p) < - alpha*c*m:
        alpha *= tau
    return alpha


def bfgs_w_backtrack(X_0, obj_func, obj_func_grad,
                     max_iter=500,
                     tol=1e-5,
                     alpha=1,
                     tau=0.7,
                     c=1e-4):
    X_0 = X_0.copy()
    B_0 = np.eye(X_0.shape[0])
    gradient_0 = obj_func_grad(X_0)
    steps = [X_0.copy()]
    for i in range(max_iter):
        p = -B_0 @ gradient_0
        alpha = line_search_armijo(X_0, p, obj_func, obj_func_grad, alpha,
                                   tau, c)
        X_1 = X_0 + alpha * p
        if np.allclose(X_0, X_1, tol):
            break
        gradient_1 = obj_func_grad(X_1)
        s_0 = X_1 - X_0
        y_0 = gradient_1 - gradient_0
        B_1 = B_0 \
            + (y_0 @ y_0.T)/(y_0.T @ s_0) \
            - (B_0 @ s_0 @ s_0.T @ B_0)/(s_0.T @ B_0 @ s_0)
        X_0 = X_1
        B_0 = B_1
        gradient_0 = gradient_1
        steps.append(X_0.copy())

    return X_0, obj_func(X_0), steps
```

In [ ]:

```
def trust_region(p, delta):
    norm2_p = norm2(p)
    if norm2_p <= delta:
        return p
    else:
        return p * delta / norm2_p


def bfgs_w_trust_region(X_0, obj_func, obj_func_grad,
                        max_iter=500,
                        tol=1e-5,
                        delta_max=.01,
                        eta=1e-2):
    n = len(X_0)
    B_0 = np.eye(n)
    X_0 = np.array(X_0).astype(float)
    delta = delta_max
    steps = [X_0.copy()]
    for i in range(max_iter):
```

```
        gradient_0 = obj_func_grad(X_0)
        p = -B_0 @ gradient_0
        p = trust_region(p, delta)

        rho = (obj_func(X_0) - obj_func(X_0 + p)) / (-gradient_0.T @ p - 0.5 * p.T

        if rho < 0.25:
            delta *= 0.25
        elif rho > 0.75 and norm2(p) == delta:
            delta = min(2 * delta, delta_max)

        if rho > eta:
            X_1 = X_0 + p
            if np.allclose(X_0, X_1, tol):
                break
            gradient_1 = obj_func_grad(X_1)
            s_0 = X_1 - X_0
            y_0 = gradient_1 - gradient_0
            B_1 = B_0 \
                + (y_0 @ y_0.T)/(y_0.T @ s_0) \
                - (B_0 @ s_0 @ s_0.T @ B_0)/(s_0.T @ B_0 @ s_0)
            X_0 = X_1
            B_0 = B_1
            gradient_0 = gradient_1
            steps.append(X_0.copy())

    return X_0, obj_func(X_0), steps
```

Due to the issue that the algorithms get stuck at a local minima, a multirandom initialization method is implemented. The one yielding the best results is then selected.

In [ ]:
```
def multi_start_bfgs_w_backtrack(starting_points, obj_func, obj_func_grad,
                                 max_iter=500,
                                 tol=1e-5,
                                 alpha=1,
                                 tau=0.7,
                                 c=1e-4):
    best_sol = None
    best_val = np.inf
    steps_for_best = []
    for X_0 in starting_points:
        result, func_val, steps = bfgs_w_backtrack(
            X_0,
            obj_func,
            obj_func_grad,
            max_iter,
            tol,
            alpha,
            tau,
            c
        )
        if func_val < best_val:
            best_sol = result
            best_val = func_val
            steps_for_best = steps
```

```
    return best_sol, best_val, steps_for_best


def multi_start_bfgs_w_trust_region(starting_points, obj_func, obj_func_grad,
                                    max_iter=500,
                                    tol=1e-5,
                                    delta_max=.01,
                                    eta=1e-2):
    best_sol = None
    best_val = np.inf
    steps_for_best = []
    for X_0 in starting_points:
        result, func_val, steps = bfgs_w_trust_region(
            X_0,
            obj_func,
            obj_func_grad,
            max_iter,
            tol,
            delta_max,
            eta
        )
        if func_val < best_val:
            best_sol = result
            best_val = func_val
            steps_for_best = steps
    return best_sol, best_val, steps_for_best
```

Function to visualize the trajectory of descent.

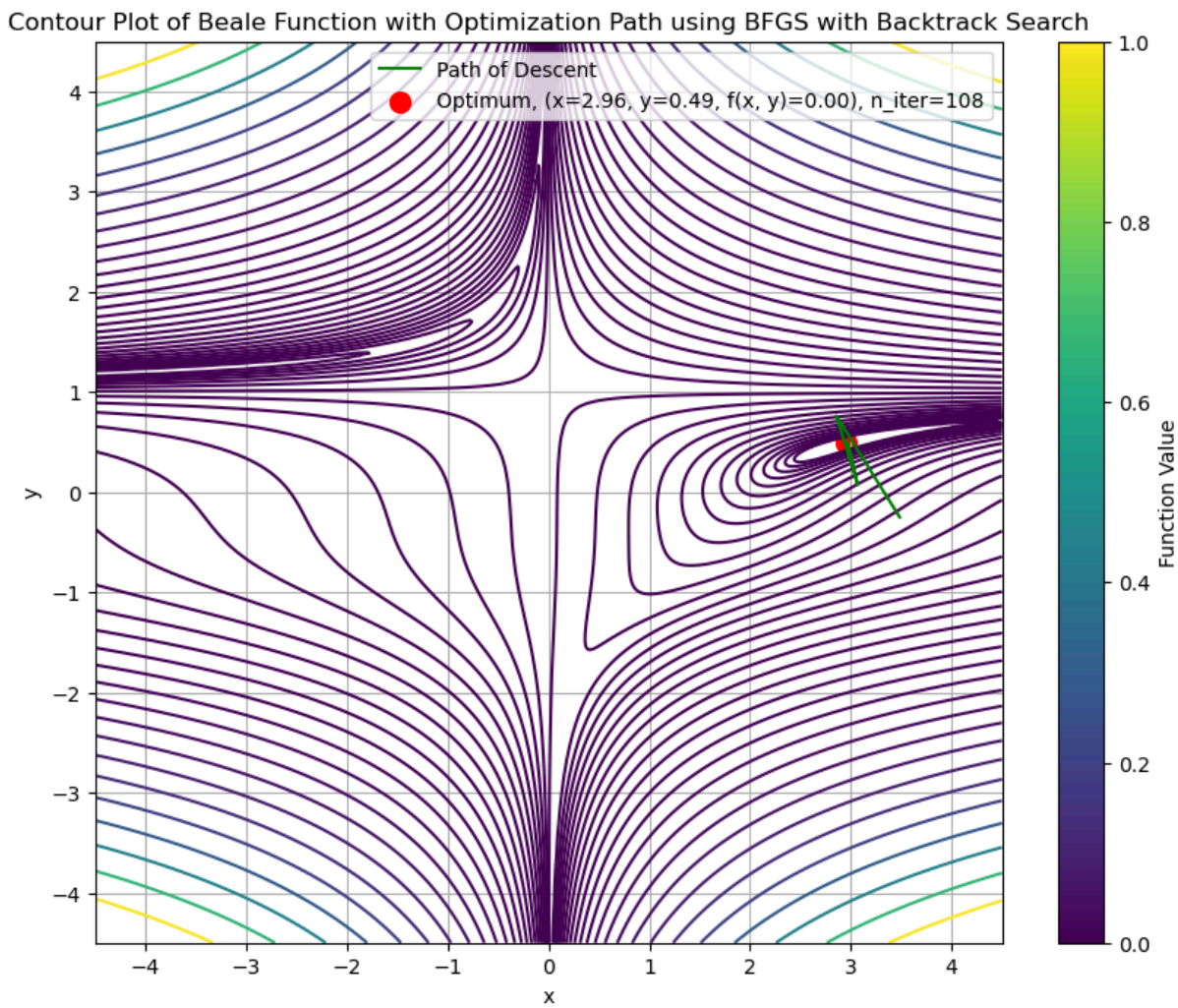
In [ ]:

```
def plot_descent_path(results, func_val, steps, algo_name):
    steps = np.array(steps)

    x = np.linspace(-4.5, 4.5, 400)
    y = np.linspace(-4.5, 4.5, 400)
    x, y = np.meshgrid(x, y)
    z = obj_func([x, y])

    plt.figure(figsize=(10, 8))
    plt.contour(x, y, z, levels=np.logspace(-1, 5, 35), cmap='viridis')
    plt.plot(steps[:, 0], steps[:, 1], 'g-', markersize=5, linewidth=1.5, label='Pa
    plt.scatter(
        steps[-1, 0],
        steps[-1, 1],
        color='red',
        s=100,
        label=f'Optimum, (x={results[0][0]:.2f}, y={results[1][0]:.2f}, f(x, y)={fu
    )
    plt.colorbar(label='Function Value')
    plt.title(f'Contour Plot of Beale Function with Optimization Path using {algo_n
    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend()
    plt.grid(True)
```

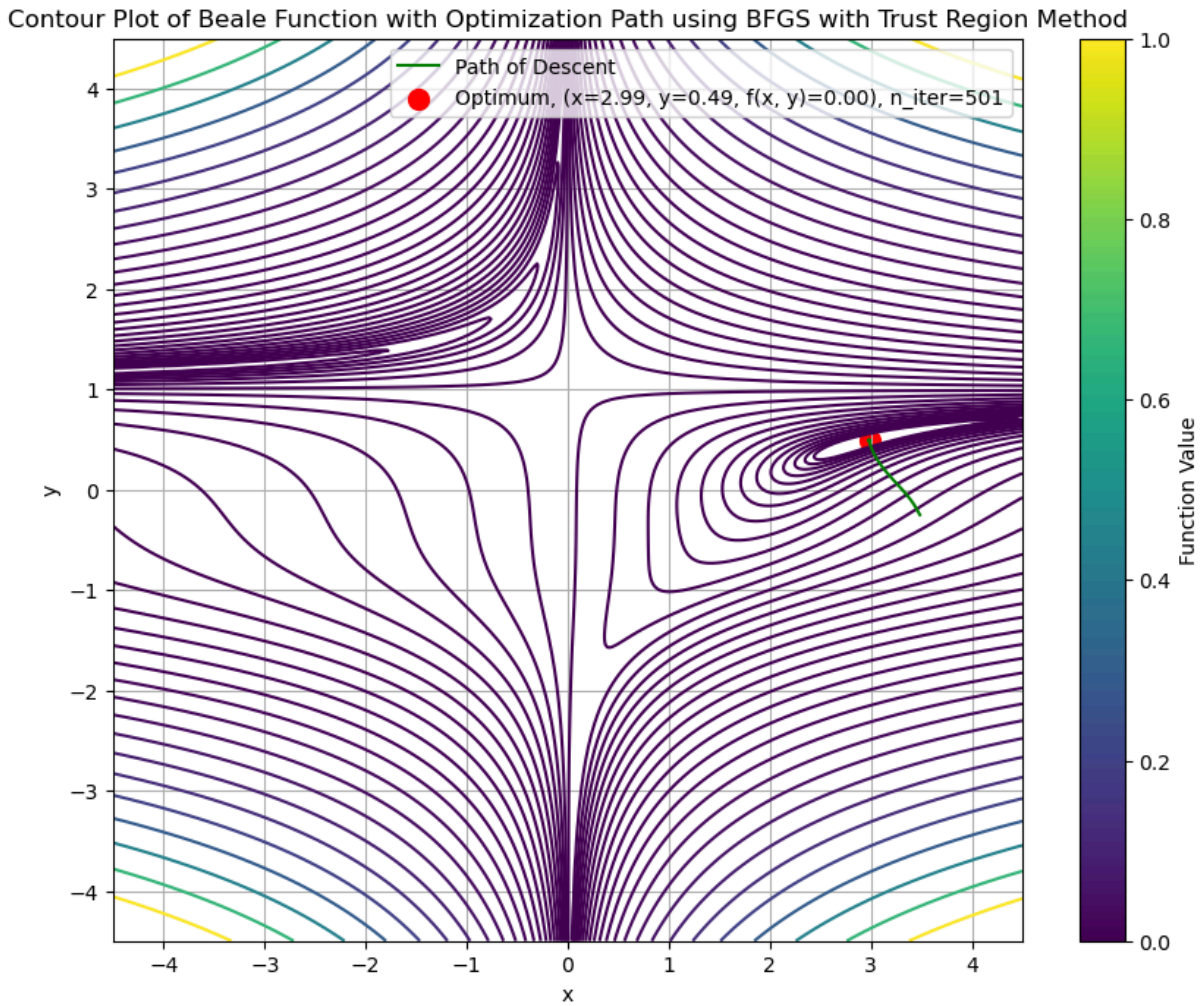
Of course the answer to life, the universe, and everything, is used as a seed.

In [ ]:
```
np.random.seed(42)
starting_points = [np.random.uniform(-4.5, 4.5, (2, 1)) for _ in range(50)]
```

In [ ]:
```
plot_descent_path(
    *multi_start_bfgs_w_backtrack(starting_points, obj_func, obj_func_grad),
    'BFGS with Backtrack Search'
)
```



In [ ]:
```
plot_descent_path(
    *multi_start_bfgs_w_trust_region(starting_points, obj_func, obj_func_grad),
    'BFGS with Trust Region Method'
)
```


Contour Plot of Beale Function with Optimization Path using BFGS with Trust Region Method
Path of Descent
Optimum, (x=2.99, y=0.49, f(x, y)=0.00), n_iter=501
x
y
Function Value